



Instituto **Tecnológico**[®]
de Aguascalientes



INSTITUTO TECNOLÓGICO DE AGUASCALIENTES

Integrante:

Manuel Humberto Reyes Salas

María José Guerrero Rios

Jesús Eli Sánchez Ruvalcaba

Actividad:

Materia: Tecnologías inalámbricas

Docente: RICARDO ALEJANDRO RODRIGUEZ JIMENEZ

Fecha: 18 de Mayo del 2026

1. OBJETIVO DE LA PRÁCTICA

El objetivo primordial de esta actividad consiste en diseñar, configurar e implementar un Portal Cautivo totalmente funcional utilizando el microcontrolador ESP32 operando en modo Access Point (AP) nativo. El sistema debe ser capaz de emitir una señal de red inalámbrica Wi-Fi bajo un Identificador de Conjunto de Servicios (SSID) personalizado. Al momento en que cualquier dispositivo cliente (teléfono inteligente, tableta o computadora) intente asociarse a dicha red inalámbrica, los subsistemas del microcontrolador deben interceptar de manera obligatoria las peticiones de resolución de nombres y redireccionar el tráfico de forma automática hacia un servidor web embebido local en el dispositivo.

Para cumplir rigurosamente con los requerimientos didácticos y de ingeniería establecidos, la solución debe incorporar las siguientes especificaciones obligatorias:

- Habilitación del hardware de radiofrecuencia de la ESP32 para inicializar y sostener un punto de acceso independiente.
- Difusión activa de un SSID específico que actúe como la red de control inalámbrico del proyecto.
- Despliegue automático de una interfaz gráfica web (Portal Cautivo) sin necesidad de intervención explícita o digitación de URLs por parte del usuario final.
- Temática libre orientada a servicios tecnológicos reales; en este escenario, se ha seleccionado la arquitectura de un 'Sistema de Acceso Inteligente'.
- Formulario de captura embebido que exija la introducción obligatoria de al menos dos datos del usuario (Nombre Completo y Número de Control Institucional) acordes a la temática seleccionada.
- Presentación de información del sistema pertinente dentro de la página raíz, detallando el estado operativo del microprocesador.
- Generación de una pantalla de respuesta dinámica que brinde un mensaje explícito de bienvenida y muestre el eco de los datos capturados tras un registro exitoso.

2. DESCRIPCIÓN DE LA TECNOLOGÍA

2.1 Arquitectura de la ESP32 en Modo Access Point (SoftAP)

A diferencia del modo Estación (STA), donde el dispositivo actúa como un cliente que se asocia a una infraestructura de red inalámbrica gobernada por un enrutador comercial, el modo SoftAP (Software-enabled Access Point) instruye al controlador de red de la ESP32 para emular el comportamiento completo de un punto de acceso central. En este modo, el chip gestiona de forma autónoma las capas físicas y de enlace de datos (IEEE 802.11), administrando las balizas de sincronización (Beacons), la autenticación de clientes, la asignación de ranuras de tiempo y la distribución de direccionamiento IP básico dentro de un rango de subred local privado y aislado.

2.2 Mecanismo del Servidor DNS para la Intercepción de Tráfico

El núcleo lógico que permite la existencia de un portal cautivo radica en la manipulación del Sistema de Nombres de Dominio (DNS). Cuando un sistema operativo moderno se conecta a una red Wi-Fi, inmediatamente despacha peticiones DNS silenciosas en el puerto UDP 53 hacia dominios propietarios (como connectivitycheck.gstatic.com en Android o captive.apple.com en iOS) con el fin de validar el acceso real a la red WAN de Internet. Al levantar un servicio DNS local mediante la clase `DNSServer` dentro de la ESP32, configurado con un comodín de resolución asterisco (*), el dispositivo captura de manera indiscriminada cualquier consulta DNS de texto y responde de vuelta con su propia dirección IP estática de pasarela. Esto engaña al dispositivo cliente, forzándolo a abrir la interfaz de navegación integrada del sistema operativo apuntando directamente hacia el servidor web local.

2.3 Protocolo HTTP y Gestión de Servidores Web Embebidos

La capa de aplicación y presentación está sustentada en el protocolo HTTP/1.1 a través de un socket de escucha en el puerto TCP de hipertexto estándar (Puerto 80). La clase `WebServer` de la arquitectura del núcleo de Espressif permite asociar vectores de interrupción lógicos a rutas URI específicas. Para esta práctica, se diseñó un enrutamiento dual mediante métodos síncronos: la raíz principal (/) despacha el documento de presentación inicial a través de peticiones GET, mientras que la subruta de procesamiento (/guardar) atiende de forma exclusiva transacciones bajo el método POST, garantizando el aislamiento de los datos del formulario dentro del cuerpo del mensaje HTTP en lugar de exponerlos explícitamente en la barra de direcciones.

3. ESPECIFICACIONES TÉCNICAS Y CONEXIÓN

A nivel de hardware, el proyecto saca provecho del SoC integrado y la antena de PCB de la ESP32, por lo que no requiere de sensores periféricos acoplados físicamente a los pines GPIO, a diferencia del sistema previo basado en el bus SPI y el lector PN532. La alimentación se realiza directamente por el bus USB a 5V o mediante pines dedicados de voltaje regulado. La configuración de los parámetros de la infraestructura lógica de red se describe minuciosamente en la siguiente tabla:

Parámetro de Red	Valor / Configuración Asignada
Identificador Wi-Fi (SSID)	ControlNFC_ESP32
Dirección IP Estática (Gateway)	192.168.4.1
Máscara de Subred (Subnet Mask)	255.255.255.0
Puerto del Servidor DNS	Puerto 53 (UDP)
Puerto del Servidor HTTP Web	Puerto 80 (TCP)

4. CÓDIGO FUENTE IMPLEMENTADO Y EXPLICACIÓN DETALLADA

El firmware ha sido desarrollado bajo el entorno de Arduino Core para ESP32, implementando un paradigma estructurado y modularizado mediante funciones de callback. El código completo y subdividido estructuralmente se presenta a continuación:

4.1 Directivas de Inclusión, Objetos Globales y Variables

```
#include <WiFi.h>
#include <WebServer.h>
#include <DNSServer.h>

const char* ssid = "ControlNFC_ESP32";

IPAddress apIP(192, 168, 4, 1);

DNSServer dnsServer;
WebServer server(80);

String nombre = "";
String control = "";
```

Líneas 1-3: Explicación de las líneas clave:

Efectúan la carga en el preprocesador de las clases que gobiernan el driver del chip Wi-Fi (`WiFi.h`), el despachador de sockets HTTP (`WebServer.h`) y el resolutor de nombres de dominio de bajo nivel (`DNSServer.h`).

Línea 7: Definición de direccionamiento IP estático corporativo. Al instanciar la clase `IPAddress` con los octetos de red especificados, se fija el nodo raíz de la subred en el identificador clásico de desarrollo inalámbrico.

Líneas 9-10: Instanciación global de los objetos de control de hilos de red. `server(80)` reserva el puerto HTTP estándar, asegurando que cualquier navegador que intente peticiones sin puerto explícito sea enrutado a los listeners locales.

4.2 Inicialización del Sistema (Rutina setup)

```
void setup() {
  Serial.begin(115200);

  WiFi.softAP(ssid);
  WiFi.softAPConfig(apIP, apIP, IPAddress(255, 255, 255, 0));

  dnsServer.start(53, "", apIP);

  server.on("/", paginaPrincipal);
  server.on("/guardar", HTTP_POST, guardarDatos);

  server.onNotFound([]() {
    server.sendHeader("Location", "/", true);
    server.send(302, "text/plain", "");
  });
}
```

```
});  
  
server.begin();  
Serial.println("Portal cautivo iniciado");  
Serial.println(WiFi.softAPIP());  
}
```

Líneas 16-17: Explicación de las líneas clave:

Ordena al firmware conmutar el estado físico de la antena interna de la ESP32 a modo Access Point. Inmediatamente después, `softAPConfig` sobrescribe los valores de IP de compuerta de enlace, servidor DHCP base y máscara de red de 24 bits, omitiendo cualquier dependencia de infraestructura externa.

Línea 19: Inicializa el demonio de resolución DNS en el puerto estándar UDP 53. El operador comodín asterisco (`\*`) representa el elemento central del Portal Cautivo: le indica al servidor DNS que sin importar cuál sea el host consultado por el cliente, siempre devuelva la dirección IP del AP (`192.168.4.1`).

Líneas 21-22: Establecen la matriz de enrutamiento del servidor web. La línea 21 mapea la raíz `/` a la función que dibuja el formulario web, mientras que la línea 22 restringe la ruta `/guardar` para que solo acepte peticiones HTTP estructuradas mediante verbos POST, ofreciendo control de seguridad sobre el flujo.

Líneas 24-27: Define el método de contingencia crítico de captura. Cuando el navegador del dispositivo cliente intenta una petición a una URL externa (por ejemplo, google.com/generate_204), la petición no coincide con ninguna ruta interna registrada. Esta función anónima intercepta el error 404 habitual, inyecta un encabezado HTTP de redirección clásica código `302 Moved Temporarily` y fuerza al cliente a cargar la raíz (`/`) del portal de forma inmediata.

4.3 Ciclo de Procesamiento Continuo (Rutina loop)

```
void loop() {  
  dnsServer.processNextRequest();  
  server.handleClient();  
}
```

Línea 35: Explicación de las líneas clave:

Ejecuta de manera síncrona y cíclica el método de escucha UDP del servidor DNS, despachando de forma inmediata las ráfagas de solicitudes hechas por los sistemas operativos de los clientes asociados.

Línea 36: Ejecuta el planificador de tareas de la pila web HTTP. Revisa el búfer de entrada de sockets TCP, analiza las cabeceras entrantes y delega los hilos de ejecución a las funciones de callback correspondientes según la URI solicitada.

5. PROCESAMIENTO DE DATOS E INTERFAZ GRÁFICA

5.1 Diseño de la Interfaz Web en Modo Oscuro (paginaPrincipal)

La construcción visual del Portal Cautivo se gestiona dinámicamente almacenando el código fuente plano de la interfaz gráfica dentro de la memoria Flash del SoC mediante macros de literales de cadena (`R"rawliteral("")`) de C++. La función encargada de despacharla es la siguiente:

```
void paginaPrincipal() {
    String pagina = R"rawliteral(
    <!DOCTYPE html>
    <html>
    <head>
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <title>Portal Cautivo</title>
        <style>
            body{ font-family: Arial; background:#0f172a; color:white; text-align:center; margin-top:40px; }
            .contenedor{ background:#1e293b; width:320px; margin:auto; padding:20px; border-radius:15px; }
            input{ width:90%; padding:10px; margin:10px; border:none; border-radius:10px; }
            button{ background:#2563eb; color:white; border:none; padding:12px; width:95%; border-radius:10px;
font-size:16px; }
            h1{ color:#38bdf8; }
        </style>
    </head>
    <body>
    <div class='contenedor'>
        <h1>Bienvenido</h1>
        <h2>Sistema de Acceso Inteligente</h2>
        <p>Ingrese sus datos para registrarse</p>
        <form action='/guardar' method='POST'>
            <input type='text' name='nombre' placeholder='Nombre' required>
            <input type='text' name='control' placeholder='Numero de Control' required>
            <button type='submit'>Registrar</button>
        </form>
        <hr>
        <p>ESP32 funcionando correctamente</p>
        <p>WiFi Activa | Modo AP habilitado</p>
    </div>
    </body>
    </html>
    )rawliteral";

    server.send(200, "text/html", pagina);
}
```

Estilización Avanzada CSS: Análisis de la Interfaz:

Se adoptó un esquema de diseño moderno basado en una paleta cromática de modo oscuro ('Dark Mode') utilizando grises profundos y azules eléctricos (`#0f172a` y `#1e293b``) que reduce la fatiga visual. Los campos de texto y los botones aprovechan pseudo-clases de redondeado de bordes (`'border-radius: 10px``) alineados con las tendencias modernas de diseño UI/UX de frameworks como TailwindCSS.

Responsividad Móvil (`viewport`): La etiqueta meta de escala se vuelve obligatoria debido a que la totalidad de los clientes que forzarán el despliegue del portal cautivo corresponden a terminales móviles. Esto asegura un renderizado responsivo adaptado al ancho físico de la pantalla del terminal.

Estructura del Formulario HTTP: Se declaran los atributos de control `name='nombre'` y `name='control'` vinculados semánticamente a un método de envío POST que apunta el puntero de acción a la subruta `/guardar`. El atributo nativo HTML5 `required` provee una validación en el lado del cliente, previniendo transmisiones nulas hacia la ESP32.

5.2 Extracción de Argumentos y Respuesta Dinámica (guardarDatos)

Al confirmarse el envío de la información por el usuario, los datos ingresados viajan codificados en el cuerpo de la transacción HTTP. El microcontrolador ejecuta la interrupción asociada a la ruta de guardado, procesando los argumentos mediante la API nativa de Espressif:

```
void guardarDatos() {
    nombre = server.arg("nombre");
    control = server.arg("control");

    String respuesta = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
body{ font-family:Arial; background:#020617; color:white; text-align:center; margin-top:50px; }
.card{ background:#1e293b; width:320px; margin:auto; padding:20px; border-radius:15px; }
h1{ color:#22c55e; }
</style>
</head>
<body>
<div class='card'>
<h1>Registro Exitoso</h1>
)rawliteral";

    respuesta += "<p>Bienvenido: " + nombre + "</p>";
    respuesta += "<p>Numero de Control: " + control + "</p>";

    respuesta += R"rawliteral(
<p>Acceso permitido</p>
<p>ESP32 Portal Cautivo</p>
</div>
</body>
</html>
)rawliteral";

    server.send(200, "text/html", respuesta);
}
```

Extracción de Parámetros (`server.arg`): Análisis del Procesamiento de Datos:

El firmware invoca el motor de parseo interno de la clase `WebServer`. Al llamar a `server.arg("nombre")`, el microcontrolador escanea las llaves del cuerpo de la petición POST

que coincidan con el identificador del campo, mapeando la cadena de caracteres recuperada y almacenándola de forma permanente en la variable global tipo objeto `String` del entorno.

Concatenación Dinámica de HTML: La variable `respuesta` actúa como un búfer dinámico de salida en memoria RAM. Utilizando la sobrecarga del operador de adición (`+=`), el hilo de ejecución concatena el código estático de la tarjeta web con los valores vivos de las variables de control del firmware, generando un documento HTML totalmente personalizado en tiempo real previo a su transmisión.

6. CONCLUSIONES INDIVIDUALES

Manuel Humberto Reyes Salas: A través de esta práctica avanzada de ingeniería de firmware, se asimiló la diferencia estructural entre las topologías de infraestructura inalámbrica. Se comprendió de forma matemática y práctica cómo el microcontrolador ESP32 puede emular un punto de acceso (SoftAP) autónomo capaz de orquestar transacciones DHCP y administrar sockets de red inalámbricos estables sin depender en absoluto de enrutadores o servidores perimetrales externos.

María José Guerrero Rios: El desarrollo me permitió asimilar a bajo nivel la estrecha relación de dependencia que existe entre los protocolos UDP de resolución de nombres (DNS) y el posterior despliegue de capas de aplicación HTTP. Lograr configurar la interceptación indiscriminada mediante el comodín de asterisco (*) en el puerto 53 expandió mis conocimientos sobre el comportamiento de seguridad de los sistemas operativos móviles y cómo estos disparan de forma automatizada los asistentes de navegación de los portales cautivos comerciales.

Jesús Eli Sánchez Ruvalcaba: Consolidé mis habilidades en el desarrollo e integración de interfaces web embebidas eficientes. Comprendí las ventajas técnicas de implementar el envío de datos mediante peticiones síncronas bajo el método POST para optimizar el búfer de entrada de la ESP32, y aprendí a consumir y parsear cadenas de datos estructurados directamente desde la RAM del SoC utilizando funciones nativas como `server.arg()`, logrando así enlazar de forma transparente la interfaz gráfica web con la lógica interna del microcontrolador.

Link del video:

<https://drive.google.com/file/d/17PztycXVpggUWCcXyf2Q1N3OE1ZXSsec/view?usp=drivesdk>

Imágenes de la práctica:

